

Validaciones obligatorias y avisos en linea

Ventas-Frontend ·

Compras-Frontend-Local

Documento de trabajo — por que un "return" silencioso confunde, y como los cinco formularios ahora avisan.

1. Que se cambio

Cinco formularios frenaban el guardado con un return y ninguna pista. El usuario apretaba Guardar, no pasaba nada, y no tenia forma de saber si el guardado habia fallado, si el boton estaba roto o si ya se habia guardado.

Ademas se pidio que un producto exija nombre, precio y stock, y que el precio y el stock no admitan el cero. La causa comun de los cinco no era cinco bugs distintos: era uno solo, y conviene entenderlo antes de leer el resto del documento.

EL PUNTO CLAVE DE TODO EL CAMBIO

Un return silencioso es indistinguible de un no-op. Para el usuario, "no guardo" y "no me avisaste que no guardo" son la misma experiencia. La seccion 2 explica por que los mensajes ya existian en el HTML y aun asi nunca aparecian.

2. La causa: los `<span>` existian, pero nadie escribia en el mapa

Los cinco formularios ya tenian el patron completo en su HTML: un span con la clase error-msg y, en productos, el input con [class.input-error]. El markup estaba bien. Lo que faltaba era el otro extremo: que el validador_shared devuelva un error cuando el campo esta vacio.

Antes de este cambio, los validadores compartidos trataban el vacio como VALIDO. Eso se ve en la firma: no tenian ningun parametro que dijera "esto es obligatorio".

validadores.ts — el codigo que hacia pasar el vacio

```
// core/utils/validadores.ts (ANTES)
validarTexto(valor, campo, max = 100) {
  const v = (valor || '').trim();

  if (!v) return ok(); // @@ <-- el vacio PASA
  if (v.length > max) return fail(`El ${campo} no puede tener más de ${max} caracteres.`);
  return ok();
}

validarRfc(valor) {
  const v = (valor || '').replace(/\s/g, '');

  if (!v) return ok(); // @@ <-- idem
  ...
}
```

Con ese comportamiento, la cadena completa de un formulario era:

categorias.ts — el `<span>` existed, pero `r.ok` siempre era true con el campo vacio

```
validarNombre() {
  const r = validarTexto(this.categoryName, 'nombre de la categoría', 50);
  if (r.ok) delete this.errores['name']; // @@ borra el error...
  else      this.errores['name'] = r.error;
}

saveCategory() {
  if (!this.categoryName.trim()) return; // @@ ...y despues frena en silencio
}
```

POR ESO EL MENSAJE "NO APARECIA"

El HTML estaba correcto y el validador tambien, por separado. El que fallaba era el encadenamiento: validar devolvía ok con el vacío, el delete se llevaba el error, y el span se quedaba con la condición falsa. No era un problema de CSS ni de Angular.

3. Los flags que faltaban en validadores.ts

La corrección fue agregar parámetros opcionales. Todos tienen un valor por defecto que conserva el comportamiento anterior, para no romper los doce servicios que ya los usan.

Las seis firmas después del cambio

```

validarTexto(valor, campo, max = 100, obligatorio = false)
validarEmail(valor, obligatorio = false)
validarPassword(valor, obligatorio = false)
validarRfc(valor, obligatorio = false)
validarEntero(valor, campo, min = 0)
validarPrecio(valor, campo, min = 0)

```

El flag obligatorio se resuelve con una sola funcion auxiliar que decide el genero del mensaje, porque "La contrasena es obligatorio" esta mal gramaticalmente y un detalle asi se nota.

validadores.ts — el flag obligatorio

```

function mensajeObligatorio(campo: string, femenino = false): string {
  return femenino
    ? `La ${campo} es obligatoria.`
    : `El ${campo} es obligatorio.`;
}

export function validarTexto(valor: string, campo: string, max = 100, obligatorio = false) {
  const v = (valor || '').trim();

  if (!v) {
    return obligatorio
      ? fail(mensajeObligatorio(campo, campo === 'contraseña'))
      : ok(); // @@ sin flag, sigue siendo opcional
  }
  ...
}

```

Con eso, los cinco formularios pasaron a exigir lo que corresponde:

FORMULARIO	CAMPO	FIRMA QUE SE USA
Productos	Nombre	validarTexto(v, "nombre", 80, true)
Productos	Precio	validarPrecio(v, "precio", 1)
Productos	Stock	validarEntero(v, "stock", 1)
Productos	SKU / Barcode	validarSku(v) / validarBarcode(v)
Categorias	Nombre	validarTexto(v, "nombre", 30, true)
Roles	Nombre	validarTexto(v, "nombre", 30, true)
Usuarios	Nombre	validarTexto(v, "nombre", 30, true)
Usuarios	Correo	validarEmail(v, true)
Usuarios	Contrasena	obligatoria solo al crear
Proveedores	Nombre	validarTexto(v, "nombre del proveedor", 100, true)
Proveedores	RFC	validarRfc(v, true)

4. Por que min = 0 sigue siendo el default en precio y stock

Agregar el min a validarEntero y validarPrecio es comodo, pero el default importa mas de lo que parece. Si se hubiera puesto min = 1 por defecto, se habria roto el formulario de caja.

El unico otro consumidor de validarPrecio

```
// features/charge/facade/cash-facade.ts
const resultado = validarPrecio(this.openingAmount, 'monto inicial');
if (!resultado.ok) { ... }
```

EL MONTO INICIAL DE CAJA PUEDE SER CERO

Una caja puede abrirse sin fondo: el cajero abre el turno y deposita despues. Con min = 1 como default, esa operacion legitima quedaria bloqueada por un cambio pensado para productos. Por eso el default se queda en 0 y es productos quien pide min = 1 de forma explicita.

La regla quedo entonces donde corresponde: en el formulario de productos, no en el servicio ni en el backend.

EL STOCK 0 ES VALIDO EN LA BASE

Un producto que se vendo hasta agotarse queda con stock 0 en la base, y ahi no es un error: es el estado normal de un producto agotado. Lo que no se permite es crearlo en 0. El min = 1 va en el formulario, no en el servicio, para no romper la fila que ya esta en la base con 0.

5. Productos: novalidate es lo que hace que el aviso aparezca

El formulario de productos paso de min="0" a min="1" en precio y stock. Eso bien, pero solo con min el resultado seria el equivocado: el navegador bloquea el submit por su cuenta y el usuario ve su burbuja nativa.

productos.html — el campo de nombre, que antes no se validaba

```
<form class="form-card" (ngSubmit)="save()" novalidate>
  <label>Nombre del producto</label>
  <input
    name="name"
    [(ngModel)]="form.name"
    (input)="validarCampo('name')"
    [class.input-error]="errores['name']"
    appSelectOnFocus maxLength="80"
  />
  <span class="error-msg" *ngIf="errores['name']">{{ errores['name'] }}</span>
</form>
```

DOS AVISOS PELEANDO POR EL MISMO SUBMIT

Sin novalidate, el min="1" del navegador detiene el formulario antes de que Angular vea nada. El usuario ve un globito gris del navegador, en ingles, que no dice que campo es el culpable. Gana el nativo porque ocurre primero, y nuestro mensaje en rojo nunca se pinta. novalidate no desactiva la regla: el min sigue sirviendo para el spinner y el teclado, solo le quita la autoridad de frenar el formulario.

Con novalidate, la autoridad de frenar el guardado pasa a ser validarCampo(), que ademas puede escribir en el mapa de errores y marcar el input.

6. El caso del nombre: dos que faltaban y uno que sobr

El campo nombre de productos no tenia ni validacion ni mensaje. Con nombre vacio, el guardado pasaba la validacion de precio, stock, SKU y barcode, y se enviaba el formulario al backend.

productos.ts — nombre entra a la lista y precio/stock piden min = 1

```
// productos.ts (ANTES) - "name" ni siquiera estaba en la lista
formularioValido(): boolean {
  (['price', 'stock', 'sku', 'barcode'] as const)
    .forEach((c) => this.validarCampo(c));
  return Object.keys(this.errores).length === 0;
}

// productos.ts (AHORA)
formularioValido(): boolean {
  (['name', 'price', 'stock', 'sku', 'barcode'] as const)
    .forEach((c) => this.validarCampo(c));
  return Object.keys(this.errores).length === 0;
}

validarCampo(campo: 'name' | 'price' | 'stock' | 'sku' | 'barcode') {
  const valor = this.valorDe(campo);

  let resultado;
  switch (campo) {
    case 'name':
      resultado = validarTexto(String(valor ?? ''), 'nombre', 80, true);
      break;
    case 'price':
      resultado = validarPrecio(valor, 'precio', 1); // @@ min = 1 explicito
      break;
    case 'stock':
      resultado = validarEntero(valor, 'stock', 1); // @@ min = 1 explicito
      break;
    ...
  }
  if (resultado.ok) delete this.errores[campo];
  else this.errores[campo] = resultado.error;
}
```

El mismo sweep del HTML quedo en los cinco formularios:

FORMULARIO	ANTES	AHORA
categorias.ts	return silencioso con el campo vacio	validarNombre() y return si errores['name']
roles.ts	return silencioso (2 veces: create y update)	validarNombre() en ambos caminos
usuarios.ts	snackbar con la contraseña corta	validarFormulario() con aviso en el campo
compras.ts	return si falta nombre o RFC	validarFormularioProveedor()
productos.ts	nombre sin validar	nombre en la lista + min = 1 en precio y stock

7. El detalle que hace funcionar la validacion en compras y usu

Dos formularios tenían dos atributos (input) en la misma etiqueta: uno para pasar a mayúsculas y otro para validar. Angular los dispara en orden de plantilla, pero ngModel actualiza su propio valor por su cuenta y sin avisar.

El problema del orden

```
<!-- compras.html (ANTES) — el RFC tenía TRES (input) -->
<input
  name="formProvider.rfc"
  [(ngModel)]="formProvider.rfc"
  (input)="aMayusculas($event)"
  (paste)="onPasteRfc($event)"
  (input)="validarCampo('rfc')" // @@ depende del orden
/>
```

Si el orden queda al revés, la validación corre contra el texto sin mayúsculas de formProvider.rfc mientras la caja ya muestra mayúsculas. El error se calcula sobre un valor que el usuario nunca vio.

compras.ts — los métodos que reemplazan los atributos múltiples

```
// compras.ts (AHORA) — un método, orden explícito
onInputRfc(event: Event) {
  aMayusculas(event); // @@ 1. mayusculas primero
  this.validarCampo('rfc'); // @@ 2. validar después
}

onInputProviderName(event: Event) {
  aMayusculas(event);
  this.validarCampo('name');
}
```

Lo mismo se aplicó a productos con onInputSku() para el campo SKU.

8. Usuarios: el snackbar era correcto pero se perdía

En usuarios el problema no era un return mudo sino el aviso equivocado: la contraseña se validaba con snackbar, y solo la contraseña. Nombre y correo se avisaban con el span pero no marcaban el input, porque no tenían el atributo que falta.

El snackbar también tapaba el formulario, se iba solo a los 4 segundos y no señalaba el campo culpable. Para la contraseña eso se nota poco; para un formulario con tres campos, el aviso tiene que estar pegado al campo.

usuarios.ts — la contraseña sigue con el mismo texto, pero en el campo

```
// usuarios.ts (AHORA)
validarPassword() {
  const pw = this.form.password ?? '';

  if (!pw) {
    if (this.form.id) {
      delete this.errores['password']; // @@ al editar, vacia = "no la cambio"
    } else {
      this.errores['password'] = 'La contraseña es obligatoria.';
    }
    return;
  }

  const largo = pw.trim().length;
  if (largo < this.MIN_PASSWORD_LENGTH) {
    this.errores['password'] = `Mínimo ${this.MIN_PASSWORD_LENGTH} caracteres (llevas ... ${largo}).`;
  } else if (largo > this.MAX_PASSWORD_LENGTH) {
    this.errores['password'] = `Máximo ${this.MAX_PASSWORD_LENGTH} caracteres (llevas ... ${largo}).`;
  } else {
    delete this.errores['password'];
  }
}
}
```

POR QUE LA CONTRASEÑA SIGUE SIENDO OBLIGATORIA AL CREAR

Al editar, updateUser no manda el campo password. Exigirla en edicion dejaria imposible guardar cualquier edicion: habria que escribir una contraseña nueva cada vez que se quisiera corregir un nombre. Por eso la regla es condicional al form.id, no una regla fija.

9. El cambio de mensaje en roles

En roles.ts el mensaje de error decia "El nombre de la categoria no puede tener mas de 50 caracteres". Era un copy-paste de categorias.ts: el campo es el nombre del rol, no de una categoria. Con el flag obligatorio y el campo corregido, ahora dice:

roles.ts — el mensaje ya no dice "categoría"

```
// roles.ts (ANTES)
validarNombre() {
  const r = validarTexto(this.roleName, 'nombre de la categoría', 50);
  ...
}

// roles.ts (AHORA)
validarNombre() {
  const r = validarTexto(this.roleName, 'nombre', 30, true);
  ...
}
```

10. Los estilos del aviso

El aviso en línea necesita dos reglas: una para el borde del input en rojo y otra para el texto debajo del campo.

Las dos estaban duplicadas en productos.css y

compras.css, con el mismo !important en cada copia.

Se dejaron de declarar en los CSS de cada componente y se movieron a styles.css, que es donde viven las demás reglas compartidas de la app (la barra de filtros, la tabla oscura, el pie de paginación). Los cinco formularios las necesitan y ninguno de los dos CSS de componente cubría a los otros tres.

styles.css — el bloque que se movió al global

```
/* styles.css — una sola vez, para los cinco formularios */

.error-msg {
  color: #ef4444;
  font-size: 12px;
  margin-top: 4px;
  display: block;
}

.input-error {
  border-color: #ef4444 !important;
  background: rgba(239, 68, 68, 0.05) !important;
}
```

El !important en el borde es a propósito. Los formularios declaran su propio border-color con la misma especificidad que .input-error, así que sin el !important ganaría el del componente y el input se vería con su color normal, con el mensaje de error debajo pero sin el campo marcado.

11. Lo que NO se tocó, y por qué

PUNTO	DECISION
Caja: monto inicial	validarPrecio se queda con min = 0; el monto inicial puede ser 0
Stock 0 en la base	Es valido. El min = 1 aplica solo al formulario de productos
Productos: validateNumber()	Se conserva pero ya no lo invoca el template (ver abajo)
Usuarios: al editar	La contraseña vacia sigue siendo valida
Categorias y roles	maxlength = 30, y el validador ahora pide max = 30 (antes 50)
Backend	Sin cambios. La obligatoriedad ya estaba ahi desde el commit a672cef

El punto de validateNumber() merece su nota. Ese metodo de productos lo invocaba el template en (input) y hacia esto:

12. El 0 del stock: el que escribe, decide

La correccion silenciosa

```
// productos.ts - el metodo que el template ya no llama
validateNumber(field: 'price' | 'stock') {
  const min = field === 'stock' ? 0 : 1;
  if (this.form[field] < min) {
    this.form[field] = min;      // @@ corrige en silencio: 0 se vuelve 1
  }
}
```

Escribir 0 en el stock y ver la caja saltando a 1 sin aviso es peor que un error de validacion: el usuario cree que escribio 1 cuando el campo muestra 1, pero el motivo no lo sabe, y el formulario no le explico nada. Se desacoplo la correccion del aviso: el template ya llama a validarCampo('stock'), que avisa y deja el 0 que el usuario escribio.

EL METODO SE CONSERVA, PERO DOCUMENTADO COMO NO USADO

No se borro porque podria servir en otra parte, pero no debe volver a engancharse al (input). La regla que queda: si un metodo corrige el valor del usuario sin avisar, no va en un evento de escritura. Si va, tiene que avisar.

13. Verificacion

QUE SE REVISO	RESULTADO
npx ng build --configuration development	OK (11.7 s, 1.68 MB)
categorias.ts / roles.ts	Nombre vacio avisa antes de guardar
usuarios.ts	Contraseña, nombre y correo avisan en su campo
compras.ts	Nombre y RFC vacios avisan antes de guardar

QUE SE REVISO	RESULTADO
productos.ts	Nombre, precio y stock avisan; min = 1
npx ng test --watch=false	No encuentra specs (ver abajo)

Los tests no corren por el entorno, no por el codigo: la ruta del repositorio tiene parentesis ("Sistema de ventas (comida)") y eso rompe el patron que busca los archivos de prueba. Es un problema conocido de esta maquina.

Revision manual sugerida, en este orden: abrir productos, apretar Guardar con el formulario vacio y confirmar que los cinco campos se ponen rojos con su mensaje debajo. Luego escribir 0 en el stock y confirmar que avisa en vez de saltarse a 1. Despues recorrer categorias, roles, usuarios y proveedores con el campo vacio y ver que aparece el mismo patron en los cuatro.

14. Una linea de regla

Un formulario que bloquea el guardado tiene que decir POR QUE y DONDE. Si el aviso no esta pegado al campo que lo provoco, el usuario tiene que adivinar cual de los tres campos es el culpable.

La validacion del cliente no es la garantia real: esa es del backend, y ya existe. Lo que hace el frontend es senalar donde esta el error para que el usuario no tenga que descubrirlo por eliminacion.